Experiment 6

Creation and Deployment of Smart Contract and Transactions using Solidity

Aim

To understand and deploy the `Ballot` voting smart contract (the Remix IDE sample), changing the proposal names from `bytes32` to `string`, and to exercise its full flow of transactions: giving the right to vote, delegating, voting, and declaring a winner.

Theory

`Ballot.sol` implements electronic voting with delegation. The chairperson (the deploying account) creates one `Proposal` per name and gives individual addresses the right to vote; each voter has a `weight` (1 by default, increased when others delegate to them), a `voted` flag, an optional `delegate`, and the index of the proposal they voted for. `vote()` adds the voter's weight to a proposal's `voteCount`; `winningProposal()` returns the index with the highest count, and `winnerName()` returns its name.

Change made for this experiment: the original sample stores each proposal name as `bytes32`, so deploying it in Remix needs every name converted to a hex string first (hence the web3-type-converter link in the manual). Here `Proposal.name` and the constructor argument are plain `string`, so names such as `"Proposal A: Library Upgrade"` are passed in as-is.

Viva topics

1. **Relevance of `require` statements.** `require(condition, "message")` is a guard: if the condition is false, execution stops, every state change made in that transaction is rolled back, and the message is returned to the caller. In `Ballot` it enforces who may act (`msg.sender == chairperson`, `weight != 0`) and when (`!voted`), so an invalid call can never corrupt the vote counts.

2. **`mapping`, `storage` and `memory`.** A `mapping(address => Voter)` is a hash-table-like key-value store; it has no length and cannot be iterated, but lookups are cheap. `storage` is the contract's permanent on-chain state (writes cost gas); a `storage` local such as `Voter storage sender = voters[msg.sender]` is a *reference* to that state, so edits persist. `memory` is temporary, per-call data that is discarded when the function returns, and is cheaper (e.g. the `string[] memory proposalNames` constructor argument).
3. **Why `bytes32` instead of `string`?** `bytes32` is fixed-size, fits in one storage slot and is cheap to store and compare, but limits names to 32 bytes and needs off-chain conversion for readability. `string` is dynamically sized, human-readable and unlimited in length, but costs more gas and cannot be compared with `==` directly. For a small ballot, readability outweighs the extra gas, which is why this experiment uses `string`.

Tools used: Node.js 24, Hardhat 2.29 with `@nomicfoundation/hardhat-toolbox` (bundles `ethers.js` v6), Solidity 0.8.24. Run directly in a terminal against a standalone `npx hardhat node` (port 8547), in place of the Remix IDE browser environment, since this was done without a browser wallet.

Output

1. Deploy `Ballot` with string proposal names

```
=== 1. Deploy Ballot with STRING proposal names (no bytes32 conversion needed) ===
Ballot deployed at: 0x5FbDB2315678afecb367f032d93F642f64180aa3
chairperson(): 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
proposals(0) -> name: "Proposal A: Library Upgrade", voteCount: 0
proposals(1) -> name: "Proposal B: Sports Complex", voteCount: 0
proposals(2) -> name: "Proposal C: Cafeteria Renovation", voteCount: 0
```

The contract is deployed with three human-readable proposal names passed directly as strings (the manual's step 5), and each is read back unchanged from `proposals(i)`.

2. Give the right to vote, then delegate

```
=== 2. Chairperson gives right to vote ===
giveRightToVote(Alice) tx 0x2ca7f5dbeb14adef9b140516b6fecb7309c4b6fdff48cbfddf048ad2ab8b447e
giveRightToVote(Bob) tx 0xfde02287a46bfecf930f7b452e071f12295220c2c48158853143bdc7380f9802
giveRightToVote(Carol) tx 0x3896ae567a0e1d2459654b2f0655368c1233334cc2b75b8e58ab218480519102
voters(Alice).weight = 1
voters(Dave).weight  = 0 (never given the right)

=== 3. Delegation: Carol delegates her vote to Bob (Bob's weight becomes 2) ===
voters(Bob).weight = 2
voters(Carol).voted = true
```

The chairperson enfranchises Alice, Bob and Carol (Dave is deliberately left out). Carol's `delegate(Bob)` marks her as having voted and raises Bob's weight to 2.

3. Vote and declare the winner

```
=== 4. Voting ===
Alice voted for proposal 0
Bob voted for proposal 1 (carrying Carol's delegated vote, weight 2)
Chairperson voted for proposal 1
proposals(0) "Proposal A: Library Upgrade" -> voteCount 1
proposals(1) "Proposal B: Sports Complex" -> voteCount 3
proposals(2) "Proposal C: Cafeteria Renovation" -> voteCount 0

=== 5. Winner ===
winningProposal() = 1
winnerName()      = "Proposal B: Sports Complex"
```

Proposal B gets 3 votes (Bob's own, Carol's via delegation, and the chairperson's) against 1 for Proposal A, and `winnerName()` returns the winning name as a readable string with no `bytes32` decoding.

4. Negative tests: every `require` reverts and rolls back state

```
=== 6. Negative tests: require() statements revert and roll back state ===
Alice votes a second time: reverted as expected -> 'Already voted.'
Dave (no right) votes: reverted as expected -> 'Has no right to vote'
Non-chairperson (Alice) gives right to vote: reverted as expected -> 'Only chairperson can give right to vote.'
Carol delegates again after delegating: reverted as expected -> 'You already voted.'
proposal 2 voteCount still: 0 (failed txs changed nothing)
```

Four invalid calls are each rejected with the contract's own `require` message, and the vote counts are unchanged afterwards, demonstrating the guard-and-rollback behaviour from viva topic 1.

(Full console log saved as `deploy_output.log`, and the Hardhat node's own server-side log as `hardhat_node.log`, both in this folder; the contract is `contracts/Ballot.sol`.)

Conclusion

This experiment deployed the Remix `Ballot` contract to a local Hardhat blockchain, in place of Remix IDE + a Metamask account, with the proposal names changed from `bytes32` to `string`. The change removed the need for any off-chain hex conversion: names go in and come out as readable text. A full election was run: three voters were enfranchised, one delegated to another (raising the delegate's weight to 2), votes were cast, and `winnerName()` correctly returned "Proposal B: Sports Complex" with 3 votes against 1. Four negative tests (double voting, voting without the right, a non-chairperson granting the right, and delegating after already delegating)

each reverted with the exact `require` message and left the vote counts untouched, confirming how `require`, `mapping`, `storage` references and `memory` arguments combine to keep a voting contract’s state correct.